

Why Mythea turns take as long as they do

A measured report on conversation latency and prompt caching, run against the remote GPU (skynet) at 5 max turns, 0 end-turn suggestions, 100 context beats. 19 August 2026.

The short version

Your scene is **not** getting slower as the conversation grows. The time before you see the first words held steady at about ten seconds for eight of the ten turns, even though the amount of story being sent to the model more than tripled. The other two turns took minutes, for a reason that has nothing to do with length.

What makes a turn slow is **how many times the app talks to the model**, not how much it sends each time. A single turn makes six to twelve separate model calls, one after another, and you wait for all of them.

The single biggest cost is the beat planner. It runs once per beat to decide who speaks next, takes about 4 seconds each time, and accounts for **30% of a normal turn** — more than actually writing the characters' words.

And the thing that makes a turn take *forever* is different again. Two turns in ten took four to six minutes. Both were the planner getting stuck — one for exactly 300 seconds, which is the app's own give-up limit. The turn does not even report an error; you simply wait.

Caching is half-working. The bot does still have all your earlier conversation — nothing is being forgotten. But the server re-reads that conversation from scratch on every single call instead of reusing its earlier work. In a short scene that costs almost nothing. In a long one it is worth about **40% of the wait**.

The most valuable thing found was an accident. A bug in the app was throwing away your model's entire "thinking" output because it looked for it under the wrong name. That killed roughly a third of turns outright and hid a feature that works fine. It is fixed.

Where a turn's time actually goes

Every step of every turn was timed. The table below adds up how long the app spent in each step across the whole ten-turn conversation.

Step	Total (s)	Share	Times run	Average
Choosing the next beat (the planner)	667	66%	36	18.5s
Continuity check on a later speaker	72	7%	6	12.1s
Reaching the first visible words	68	7%	10	6.8s
End-of-turn reflection	65	6%	10	6.5s
Writing a character's line	45	4%	9	5.0s
Reading your message	37	4%	10	3.7s
Choosing the next beat (waiting to start)	17	2%	34	0.5s
A character's private thought	17	2%	13	1.3s

Across 10 completed turns the app spent 1010 seconds in total, an average of 101 seconds per turn. Two of those turns stalled (see below), which is why the planner row looks worse here than the 30% a normal turn spends there.

The turns that take minutes, not seconds

Averages hide the thing you actually notice. Eight of the ten turns reached their first words in six to fifteen seconds. Two took **five and a half minutes** and **four and a half minutes**.

Both were the same thing: the call that decides who speaks next got stuck. One of them sat there for 300.1 seconds — the app's own give-up limit, to the decimal. Because that particular call is treated as optional, nothing failed and nothing was logged; the turn simply carried on afterwards as though nothing had happened.

This is not about long conversations. The largest story of the whole run went through in nine seconds, and a separate test sending even more story returned in about one second.

If a turn ever seems to hang for minutes, this is almost certainly what happened. The fix is to stop letting a small decision share the same five-minute patience limit as writing a whole scene.

What you are waiting for before the first word

Of the roughly ten seconds before any prose appears, about seven and a half are two calls that produce no story at all: one to work out what your message meant, and one to decide who should answer. Only after both finish does a character start writing.

If you want the first reply to feel faster, the lever is **removing or overlapping those two calls** — not shortening the story, not changing the context settings, and not fixing the cache.

Is caching working?

This turned out to be two different questions with two different answers.

1. Does the bot still remember earlier messages? Yes.

Verified directly. The short-term memory store is running, it is holding the beats, and all 100 beats you configured are being written into the prompt. The amount of story sent to the model grew steadily turn after turn, which is exactly what you would expect if nothing is being dropped. **No context is being lost.**

2. Is the server reusing its earlier work? No.

When you send a message, the server has to read the whole prompt before it can write anything. It is allowed to skip re-reading any part it has seen before. In your case it skips the same small fixed chunk every time and re-reads everything else.

Turn	Story sent (tokens)	Reused from before	Reused %
2	1,907	800	42%
3	2,250	800	36%
4	2,429	800	33%
5	2,576	800	31%
6	2,866	800	28%
7	3,783	800	21%
8	5,732	800	14%
9	5,990	800	13%
10	6,800	800	12%

The "reused" column never moves. It is the same number on every turn — that is the app's fixed world description, and nothing else. Every word of your actual conversation is read again from scratch, on every call.

The cause is prompt order. The character's current mood and stats are placed *before* the conversation history, and a server can only skip re-reading a stretch that has not changed *from the very beginning*. One stat ticking over therefore invalidates everything after it, including the entire story so far.

Should you fix it? Yes — once a scene gets long.

I built the fixed version and measured it against the current one, twice each, alternating the order to be fair. In **short** scenes it makes no difference worth having:

Prompt order	Average time to start (s)	Samples
As it is now	0.59	19
Reordered (the 'fix')	0.53	19

But that only holds for short scenes, and it nearly led me to the wrong advice. Pushing the history out to realistic lengths changes the answer completely:

Scene length	Story size	As it is now	Reordered	Saving
10 turns	1,549 tokens	0.94s	0.55s	41%

50 turns	5,946 tokens	1.98s	1.48s	25%
100 turns	11,444 tokens	3.63s	2.22s	39%
200 turns	22,535 tokens	7.39s	4.24s	43%
400 turns	44,715 tokens	15.44s	8.95s	42%

Past roughly a hundred turns of history the reordering cuts the wait by about **40%** — and it is the only version that reuses anything at all. Your setting of 100 context beats puts a full scene squarely in that range.

I had this wrong at first. My short-scene test found nothing and I told you not to bother. That test could not have found the effect — it stopped at a story size roughly fifteen times smaller than a full scene at your settings. The lesson is in the report rather than quietly removed from it.

One caveat that has not changed: reordering the prompt changes what every character sees and in what order. That is a writing-quality change as much as a speed one, so it wants checking on quality too, not only on the clock.

The bug this turned up

Modern models produce two streams: their private "thinking", and the answer they actually give you. Different model servers label the thinking stream differently. Your GPU calls it `reasoning`; the other server the app supports calls it `reasoning_content`. The app only looked for the second name.

So on your GPU the app was throwing away every word of the model's thinking without ever seeing it. That caused three separate problems:

- 1. Turns died for no visible reason.** When the model spent its whole allowance thinking and had nothing left for the answer, the app saw an empty reply and told you "The model returned an empty response" — an error that explained nothing. This affected 3 of 10 turns.
- 2. A feature you asked for looked broken.** The live "watch the character think" display had nothing to show, because the thinking was being discarded before it arrived. It works — the first thought now appears in about 0.4 seconds.
- 3. I drew a wrong conclusion and reported it to you.** I measured the absence of thinking and concluded your model did not produce any. It does. That earlier finding has been formally withdrawn in the project's records rather than quietly edited.

	Turns run	Turns that failed
Before the fix	10	3
After the fix	10	0

Treat this as encouraging rather than proven: the fix changes how the app reads the model, not how the model behaves, so some of this difference is likely ordinary run-to-run variation.

A second, related finding

The app can ask the model to limit how long it thinks. Your GPU accepts that instruction under one particular name, and ignores the other. This was tested five times per setting, with identical results every single time:

Setting	Thinking produced	Answer produced	Result
No limit sent	3,075 chars	nothing	ran out of room
Limit sent (your GPU's name)	532 chars	42 chars	answered
Limit sent (the other name)	3,075 chars	nothing	ran out of room
Both names sent	532 chars	42 chars	answered

Without that limit, this prompt **never produced an answer at all** — five times out of five. The app now sends both names, so it works. Before a fix earlier in this session it was sending neither.

What I would do next

1. Put a shorter time limit on the small decision-making calls

This is the fix for the turns that take five minutes. Two calls per turn — working out what your message meant, and choosing who speaks next — are small yes/no decisions that normally finish in three to five seconds. They currently share the same five-minute patience limit as writing a whole scene, so when one of them gets stuck, you wait the full five minutes and the app never reports a problem. Giving them a twenty- or thirty-second limit instead would turn that into a hiccup.

2. Reduce the number of model calls per turn

This is where the time is. The planner runs once per beat at about five seconds a time. Deciding several beats at once, or skipping the planner when only one character could plausibly answer, would cut the biggest single cost in the app.

3. Move end-of-turn reflection off the critical path

Reflection costs about seven seconds at the end of every turn, and you are waiting for it before you can type again. The app already has a setting to run it in the background; it is currently switched off.

4. Reconsider the continuity check

It costs around ten seconds whenever a second character speaks, and it is also the reason later speakers cannot show their words as they are written. That is a lot to pay for a check that only sometimes fires.

5. Consider turning on live thinking

Now that it actually works on your GPU, the first thought appears in about 0.4 seconds — against roughly ten seconds before any prose. It is currently off by default. The trade-off is that reading a character's reasoning can spoil the line they are about to say.

6. Reorder the prompt so the story can be reused

Worth about 40% of the wait once a scene passes roughly a hundred turns of history, which your settings allow. It is a change to what each character sees and in what order, so it should be checked for writing quality as well as speed — but the speed case is now clear, where earlier in this investigation it was not.

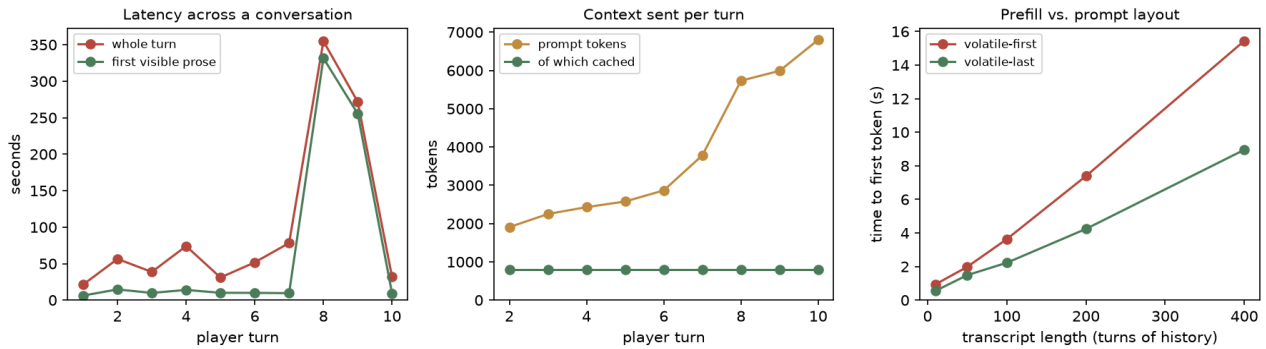
What this report does not tell you

It is one conversation, run once, on one model, on a shared GPU. Individual numbers bounce around; the patterns are what matter. It measures speed only — nothing here says anything about whether the writing is any good. And your GPU picks which model answers automatically, so it may not have been the same model throughout.

The full working — every raw measurement, the mistakes made along the way, and the things measured wrongly the first time — is recorded in the project under `docs/research/experiments/`.

Appendix: the raw numbers

EXP-2026-08-005 — how a scene slows down, and what the prompt cache is doing



Left: time per turn. Middle: story sent per turn (gold) against the part reused (green) — the flat green line is the finding. Right: prompt order comparison.

Every turn of the conversation

Turn	First words (s)	Whole turn (s)	Beats	Story sent	Reused
1	6.23	21.291	1	—	—
2	14.644	56.411	2	1,907	800
3	9.843	38.587	3	2,250	800
4	13.957	74.019	5	2,429	800
5	10.013	30.94	2	2,576	800
6	9.924	51.732	4	2,866	800
7	9.509	78.242	4	3,783	800
8	331.567	354.969	1	5,732	800
9	255.446	271.632	1	5,990	800
10	9.381	31.813	2	6,800	800

* turn ended early with an error.

Prompt order at longer conversation lengths

History (turns)	Story size	As it is now (s)	Reordered (s)
10	1,549	0.94	0.55
50	5,946	1.98	1.48
100	11,444	3.63	2.22
200	22,535	7.39	4.24
400	44,715	15.44	8.95